

使用 LiteRT 在 C++ 中進行裝置端圖像分割

來源：<https://developers.google.com/codelabs/litert-image-segmentation-cpp?hl=zh-tw>

步驟數：14

在本程式碼研究室中，您將使用 LiteRT 在 Android 上建構及執行 C++ 影像分割管道，並透過 CPU、GPU 和 NPU 加速，使用預先建構的 SDK 或從來源建構 LiteRT。

目錄

1. 事前準備
2. 影像分割
3. 做好準備
4. 瞭解專案結構
5. Core LiteRT C++ API
6. 建構 - 選項 A：預先建構的 LiteRT C++ SDK (CMake)
7. 建構 - 選項 B：使用 Bazel 建構 (從來源)
8. 使用運算著色器進行 GPU 加速前處理
9. CPU 推論
10. GPU 推論 (OpenCL)
11. 後置處理 - 去交錯和混合
12. 在裝置上部署及執行
13. 進階 (選用)：NPU 推論
14. 恭喜！

1. 事前準備

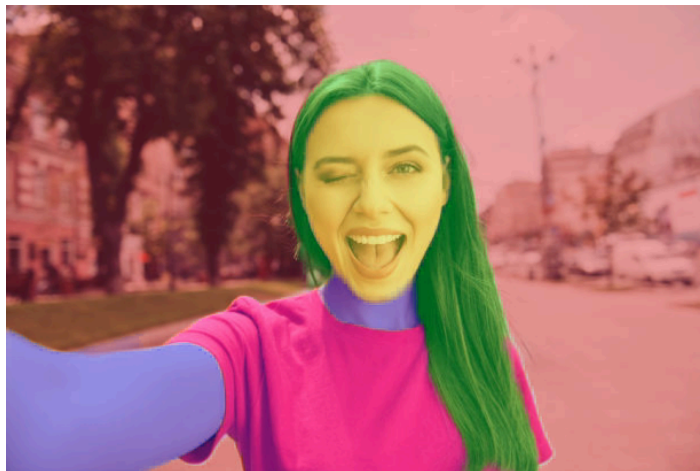
手動輸入程式碼是培養肌肉記憶的好方法，也有助於加深對教材的理解。雖然複製貼上可以節省時間，但長期而言，投入這項做法可提高效率，並增進程式設計技能。

在本程式碼研究室中，您將瞭解如何使用 Google 的高效能裝置端執行階段 **LiteRT**，建構直接在 Android 裝置上執行的 C++ 影像分割二進位檔。本程式碼研究室著重於建構 C++ 二進位檔，而非使用 Kotlin 或 Android Studio。您將使用 CMake 或 Bazel 進行交叉編譯，並透過 ADB 部署。LiteRT C++ API 適用於任何平台 (Android、Linux、嵌入式)，因此是效能關鍵應用程式、機器人和邊緣系統的實用基礎。

您將瞭解整個管道：

- 設定建構環境 (CMake + Android NDK 或 Bazel)。
- 連結 LiteRT C++ SDK，可從預先建構的版本或來源連結。
- 使用 **OpenGL ES 運算著色器**，透過 GPU 加速進行影像前處理和後處理。
- 使用 LiteRT C++ API 執行 `selfie_multiclass` 分割模型。
- 在 **CPU**、**GPU (OpenCL)** 和 **NPU (Qualcomm / MediaTek)** 上加速推論。
- 將原始模型輸出後處理為顏色混合的區隔圖片。
- 使用 ADB 部署至 Android 實體裝置，並擷取結果。

最後，您會產生類似下圖的內容：經過完整管道處理的靜態圖片，其中 6 個區隔類別分別以不同顏色疊加顯示：



必要條件

本程式碼研究室適合熟悉 C++ 的開發人員，他們想在 C++ 層級的 Android 上執行機器學習模型。您必須已經熟悉下列項目：

- C++ 基礎知識 (指標、向量、包含項目)。
- Android/ADB 基本概念 (`adb push`、`adb shell`)。
- 在 Linux 或 macOS 上使用終端機和殼層指令碼。

課程內容

- 如何使用 CMake + NDK 或 Bazel，為 Android `arm64-v8a` 交叉編譯 C++ 二進位檔。
- 如何使用 LiteRT C++ API (`Environment`、`CompiledModel`、`TensorBuffer`) 在裝置端進行高效推論。
- 瞭解 OpenGL ES 3.1 運算著色器如何完全在 GPU 上加速預先處理和後續處理。
- 如何設定 LiteRT，以透過 CPU、GPU (OpenCL) 和 NPU (Qualcomm HTP、MediaTek APU、Google Tensor) 加速。

- 同步 (`Run`) 和非同步 (`RunAsync`) 推論的差異。
- 如何使用 ADB 在 Android 上部署及執行 C++ 二進位檔。

軟硬體需求

- Linux 或 macOS 機器 (Windows 使用者應使用 WSL2)。
 - **Android NDK** r25c 以上版本 ([下載](#))。
 - **CMake** 路徑：CMake ≥ 3.22 (`sudo apt-get install cmake`)。
 - **Bazel** 路徑：已安裝 Bazel，以及完整的 LiteRT 範例存放區。
 - `PATH` (Android 平台工具) 中的 **ADB**。
 - 實體 Android 裝置，建議使用 Galaxy S24/S25 或 Pixel 進行測試。
-

2. 影像分割

影像分割是電腦視覺工作，會為圖片中的**每個像素**指派類別標籤。物件偵測會繪製定界框，而影像分割則會精確瞭解每個物件的起點和終點，達到像素級的精準度。

本程式碼研究室使用 `selfie_multiclass_256x256` 模型，將每個像素分類為 **6 個類別**之一：

類別索引	區隔
0	背景
1	美髮
2	身體肌膚
3	臉部肌膚
4	衣服
5	配件 (眼鏡、珠寶等)

模型會輸出形狀為 `[1, 256, 256, 6]` 的浮點張量。每個 256×256 像素都有 6 個信賴分數，每個類別各一個。得分最高的類別會贏得該像素 (argmax)。

LiteRT：邊緣效能

LiteRT 是 Google 的新一代高效能執行階段，適用於 TFLite 模型。透過 C++ API，您可直接存取硬體加速器，且負擔較低，並在以下三種加速器中享有一致的介面：

- **CPU**：普遍相容，在中階裝置上推論時間約為 128 毫秒。
- **GPU (OpenCL)**：推論時間約 1 毫秒；端對端時間約 17 到 43 毫秒，視緩衝區策略而定。
- **NPU**：在 Qualcomm Snapdragon、MediaTek Dimensity 9400 和 Google Tensor 裝置上，端對端延遲時間約為 9 到 28 毫秒，視 AOT 而定。JIT 編譯。

主要抽象化是 `CompiledModel`：模型會在載入時間預先編譯並針對目標硬體進行最佳化，將推論作業縮減為對預先分配緩衝區的 `Run()` 呼叫。

3. 做好準備

複製存放區

```
git clone https://github.com/google-ai-edge/litert-samples.git
```

本程式碼研究室的所有資源都位於：

```
litert-samples/compiled_model_api/image_segmentation/c++_segmentation/
```

這個目錄有兩個子專案，每個子專案都是相同範例的完整建構版本：

目錄	建構系統	LiteRT 依附元件
<code>use_prebuilt_litert/</code>	CMake + Android NDK	預先建構的 <code>litert_cc_sdk.zip</code> + <code>libLiteRt.so</code>
<code>build_from_source/</code>	Bazel	從來源編譯 LiteRT

選擇其中一個路徑並按照步驟操作。這兩個目錄的程式碼完全相同，只有建構系統和依附元件策略不同。如要以最快速度完成設定，請選擇 `use_prebuilt_litert/`。如需修改 LiteRT 本身或在現有的 Bazel 單一存放區中工作，請使用 `build_from_source/`。

檔案路徑注意事項

本教學課程中的所有檔案路徑都使用 Linux/macOS 格式。Windows 使用者應使用 WSL2。

目錄總覽

兩個子專案共用相同的來源版面配置：

```

<variant>/
|—— main_cpu.cc           # CPU inference entry point
|—— main_gpu.cc           # GPU (OpenCL) inference entry point
|—— main_npu.cc           # NPU (Qualcomm / MediaTek) entry point
|—— image_processor.h/.cc  # OpenGL ES preprocessing and postprocessing
|—— image_utils.h/.cc      # STB-based image load / save utilities
|—— timing_utils.h/.cc     # Profiling helpers
|—— shaders/              # GLSL ES 3.1 compute shaders
|   |—— preprocess_compute.glsl
|   |—— resize_compute.glsl
|   |—— mask_blend_compute.glsl
|   |—— deinterleave_masks.glsl
|   |—— passthrough_shader.vert
|—— models/
|   |—— selfie_multiclass_256x256.tflite      (CPU / GPU / NPU JIT)
|   |—— selfie_multiclass_256x256_SM8650.tflite (Qualcomm S24 AOT)
|   |—— selfie_multiclass_256x256_SM8750.tflite (Qualcomm S25 AOT)
|—— test_images/
|   |—— image.jpeg

```

此外：

- `use_prebuilt_litert/` 新增 `CMakeLists.txt`、`build_prebuilt.sh`、`deploy_and_run_on_android.sh` 和 `third_party/stb/`。
- `build_from_source/` 會新增 Bazel `BUILD` 檔案，並使用指向 `bazel-bin/` 的 `deploy_and_run_on_android.sh`。

▼ compiled_model_api

> colab

> digit_classifier

> image_classification

▼ image_segmentation

▼ c++_segmentation

> build_from_source

▼ use_prebuilt_litert

> shaders

> test_images

> third_party

\$ build_prebuilt.sh

M CMakeLists.txt

\$ deploy_and_run_on_android.sh

🔗 image_processor.cc

C image_processor.h

🔗 image_utils.cc

C image_utils.h

🔗 main_cpu.cc

🔗 main_gpu.cc

🔗 main_npu.cc

📖 README.md

🔗 timing_utils.cc

C timing_utils.h

4. 瞭解專案結構

三個進入點，一個管道

`main_cpu.cc`、`main_gpu.cc` 和 `main_npu.cc` 各包含一個 `main()` 函式，可驅動完整區隔管道。這三種管道完全相同，只有 LiteRT 加速器設定和緩衝區策略不同：

檔案	加速器	緩衝區策略
<code>main_cpu.cc</code>	<code>kCpu</code>	CPU 記憶體
<code>main_gpu.cc</code>	<code>kGpu</code> <code>kCpu</code>	使用 OpenCL 後端的 CPU 記憶體
<code>main_npu.cc</code>	<code>kNpu</code> <code>kCpu</code>	CPU 記憶體 (CPU 備援)

三者共用相同的 `ImageProcessor` (用於預先處理和後續處理的 OpenGL ES 運算著色器) 和 `ImageUtils` (STB 圖片 I/O) 公用程式。

完整管道

每個進入點都遵循相同的五階段結構：

Load → GPU upload → Preprocess (shader) → Inference (LiteRT) → Postprocess (shader) → Save

1. 載入： `ImageUtils::LoadImage()` 使用 STB 圖片程式庫將 JPEG 解碼至 CPU 記憶體。
 2. 上傳： `processor.CreateOpenGLTexture()` 將原始像素上傳至 GPU 紋理 (OpenGL RGBA8)。
 3. 前置處理 - `processor.PreprocessInputForSegmentation()` 執行 GLSL 運算著色器，將紋理大小調整為 256x256，並將像素值從 `[0, 1]` 正規化為 `[-1, 1]`。結果會存放在 GPU SSBO 中。
 4. 推論：SSBO 資料會寫入 LiteRT `TensorBuffer`，而 `compiled_model.Run()` (或 `RunAsync()`) 會執行模型。
 5. 後續處理：模型的 6 通道浮點輸出會去交錯為 6 個單通道遮罩 SSBO，然後色彩混合回原始圖片。
 6. 儲存： `ImageUtils::SaveImage()` 將最終 RGBA 圖片寫入為 PNG。
-

步驟 5 · 約 5 分鐘

5. Core LiteRT C++ API

建構前，請先熟悉所有進入點使用的三種主要 LiteRT C++ 型別。所有項目都位於 `litert::` 命名空間。

`litert::Environment`

`Environment` 是所有 LiteRT 作業的根環境。建立一次並傳遞至 `CompiledModel::Create`。如要使用 NPU，請使用供應商外掛程式庫目錄進行設定。

```
// For CPU or GPU - no extra options needed
LITERT_ASSIGN_OR_ABORT(auto env, litert::Environment::Create({}));

// For NPU: point at the vendor dispatch library directory on the device
std::vector<litert::Environment::Option> opts;
opts.push_back({litert::Environment::OptionTag::DispatchLibraryDir,
                "/data/local/tmp/cpp_segmentation_android/npv/"});
LITERT_ASSIGN_OR_ABORT(auto env,
                        litert::Environment::Create(std::move(opts)));
```

`litert::CompiledModel`

`CompiledModel` 會在建構時載入並預先編譯所要求硬體的 TFLite 模型。然後推論會減少，只剩下填滿緩衝區和呼叫 `Run()`。

```
// CPU
LITERT_ASSIGN_OR_ABORT(auto model,
                        litert::CompiledModel::Create(env, model_path,
                                                        litert::HwAccelerators::kCpu));

// GPU (pass an Options object with GpuOptions configured)
LITERT_ASSIGN_OR_ABORT(auto model,
                        litert::CompiledModel::Create(env, model_path, gpu_options));

// NPU (pass an Options object with kNpu | kCpu and vendor options)
LITERT_ASSIGN_OR_ABORT(auto model,
                        litert::CompiledModel::Create(env, model_path, npu_options));
```

`litert::TensorBuffer`

張量緩衝區會保留輸入/輸出資料。請務必從 `CompiledModel` 建立這些項目，確保大小正確，並與目標硬體對齊。

```
LITERT_ASSIGN_OR_ABORT(auto input_buffers,  
                        compiled_model.CreateInputBuffers());  
LITERT_ASSIGN_OR_ABORT(auto output_buffers,  
                        compiled_model.CreateOutputBuffers());  
  
// Write preprocessed float data, run, read results  
LITERT_ABORT_IF_ERROR(  
    input_buffers[0].Write( absl::MakeConstSpan(preprocessed_data) ));  
LITERT_ABORT_IF_ERROR(  
    compiled_model.Run(input_buffers, output_buffers));  
LITERT_ABORT_IF_ERROR(  
    output_buffers[0].Read( absl::MakeSpan(output_data) ));
```

錯誤處理巨集

巨集	行為
<code>LITERT_ASSIGN_OR_ABORT(var, expr)</code>	失敗時指派或呼叫 <code>abort()</code>
<code>LITERT_ABORT_IF_ERROR(expr)</code>	如果運算式傳回錯誤，則呼叫 <code>abort()</code>
<code>LITERT_ASSIGN_OR_RETURN(var, expr)</code>	將錯誤指派或傳播給呼叫端

步驟 6 · 約 10 分鐘

6. 建構 - 選項 A：預先建構的 LiteRT C++ SDK (CMake)

如果您不需要修改 LiteRT 本身，建議採用這個路徑。建構指令碼會處理 SDK 標頭的下載作業、複製 `.so`、擷取 STB，並透過單一指令叫用 CMake + NDK。

步驟 1：從 Maven 取得 `libLiteRt.so`

LiteRT 會在 Google Maven 的 Android AAR 中，將執行階段做為共用程式庫出貨。下載並解壓縮 `arm64-v8a` `.so`：

```
# Download the AAR
wget -O litert.aar \
    "https://dl.google.com/dl/android/maven2/com/google/ai/edge/litert/litert/2.1.3/litert-2.1.3.aar"

# Extract the runtime library
unzip litert.aar "jni/arm64-v8a/libLiteRt.so" -d extracted/
```

若要支援 GPU，請一併擷取 OpenCL/GL 加速器：

```
unzip litert.aar "jni/arm64-v8a/libLiteRtClGlaAccelerator.so" -d extracted/
```

```
aravindmurali@linux-machine:~/Documents/litert-samples/codelabs/litert-samples/compiled_model_api/image_segmentation/c++_segmentation$ wget -O litert.aar \
    "https://dl.google.com/dl/android/maven2/com/google/ai/edge/litert/litert/2.1.1/litert-2.1.1.aar"
--2026-03-31 21:13:17--  https://dl.google.com/dl/android/maven2/com/google/ai/edge/litert/litert/2.1.1/litert-2.1.1.aar
Resolving dl.google.com (dl.google.com)... 173.194.203.136, 173.194.203.93, 173.194.203.190, ...
Connecting to dl.google.com (dl.google.com)|173.194.203.136|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 7569479 (7.2M) [application/octet-stream]
Saving to: 'litert.aar'

litert.aar
100%[=====] 7.22M 34.6MB/s in 0.2s

2026-03-31 21:13:18 (34.6 MB/s) - 'litert.aar' saved [7569479/7569479]
aravindmurali@linux-machine:~/Documents/litert-samples/codelabs/litert-samples/compiled_model_api/image_segmentation/c++_segmentation$ unzip litert.aar "jni/arm64-v8a/libLiteRt.so" -d extracted/
Archive: litert.aar
  inflating: extracted/jni/arm64-v8a/libLiteRt.so
aravindmurali@linux-machine:~/Documents/litert-samples/codelabs/litert-samples/compiled_model_api/image_segmentation/c++_segmentation$
```

步驟 2：執行 `build_prebuilt.sh`

```
cd litert-samples/compiled_model_api/image_segmentation/c++_segmentation/use_prebuilt_litert/

bash build_prebuilt.sh \
    --litert_version=2.1.3 \
    --ndk_path=/path/to/android-ndk \
    --litert_so=extracted/jni/arm64-v8a/libLiteRt.so
```

指令碼會執行下列動作：

1. 從 LiteRT GitHub 版本下載 `litert_cc_sdk.zip` (SDK 標頭 + cmake 檔案) - 如果已存在，後續執行時會略過。
2. 將 `libLiteRt.so` 複製到 `litert_cc_sdk/`。
3. 將 STB 圖片標頭下載至 `third_party/stb/` (如有，則略過)。
4. 使用 Android NDK 工具鍊為 `arm64-v8a` 設定及建構 CMake，位置為 `android-26`。

成功後，您會在 `build/` 中看到三個二進位檔：

```
build/cpp_segmentation_cpu
build/cpp_segmentation_gpu
build/cpp_segmentation_npu
```

```
aravindmurali@linux-machine:~/Documents/litert-samples/code-labs/litert-samples/compiled_model_api/image_segmentation/c++_segmentation/use_prebuilt_litert$ bash build_prebuilt.sh \
--litert version=v2.1.1 \
--ndk path=/usr/local/google/home/aravindmurali/Android/Sdk/ndk/25.2.9519653 \
--litert so=extracted/jni/arm64-v8a/libLitERT.so
[ 78%] Built target absl_cordz_info
[ 79%] Built target absl_flags_config
[ 80%] Built target absl_log_internal_log_sink_set
[ 80%] Built target absl_log_initialize
[ 81%] Built target absl_cordz_sample_token
[ 81%] Built target absl_flags_internal
[ 81%] Built target absl_cord
[ 82%] Built target absl_log_internal_message
[ 83%] Built target absl_raw_hash_set
[ 85%] Built target absl_status
[ 85%] Built target absl_die_if_null
[ 85%] Built target absl_log_internal_check_op
[ 85%] Built target absl_profile_builder
[ 85%] Built target absl_flags_reflection
[ 85%] Built target absl_statusor
[ 86%] Built target absl_flags_usage_internal
[ 86%] Built target absl_hashtable_profiler
[ 86%] Built target absl_log_flags
[ 87%] Built target absl_flags_usage
[ 93%] Built target litert_cc_api
[ 94%] Built target absl_flags_parse
[ 95%] Built target timing_utils
[ 96%] Built target image_processor
[ 96%] Linking CXX executable run_model_simple
[ 96%] Linking CXX executable cpp_segmentation_gpu
[ 97%] Linking CXX executable cpp_segmentation_cpu
[ 97%] Linking CXX executable cpp_segmentation_npu
[ 98%] Built target run_model_simple
[ 98%] Built target cpp_segmentation_cpu
[ 98%] Built target cpp_segmentation_gpu
[100%] Built target cpp_segmentation_npu
Build complete. Binaries are in: /usr/local/google/home/aravindmurali/Documents/litert-samples/code-labs/litert-samples/compiled_model_api/image_segmentation/c++_segmentation/use_prebuilt_litert/build/
-rwxr-xr-x 1 aravindmurali primarygroup 15M Mar 31 21:37 /usr/local/google/home/aravindmurali/Documents/litert-samples/code-labs/litert-samples/compiled_model_api/image_segmentation/c++_segmentation/use_prebuilt_litert/build/cpp_segmentation_cpu
-rwxr-xr-x 1 aravindmurali primarygroup 15M Mar 31 21:37 /usr/local/google/home/aravindmurali/Documents/litert-samples/code-labs/litert-samples/compiled_model_api/image_segmentation/c++_segmentation/use_prebuilt_litert/build/cpp_segmentation_gpu
-rwxr-xr-x 1 aravindmurali primarygroup 15M Mar 31 21:37 /usr/local/google/home/aravindmurali/Documents/litert-samples/code-labs/litert-samples/compiled_model_api/image_segmentation/c++_segmentation/use_prebuilt_litert/build/cpp_segmentation_npu
aravindmurali@linux-machine:~/Documents/litert-samples/code-labs/litert-samples/compiled_model_api/image_segmentation/c++_segmentation/use_prebuilt_litert$
```

CMakeLists.txt 的用途

開啟 `CMakeLists.txt`。這需要 C++20，透過 `add_subdirectory` 提取 LiteRT SDK，連結 OpenGL ES 3 (`GLSV3`) 和 EGL，然後使用輔助巨集從 `main_*.cc` 來源建立每個二進位檔：

```
macro(add_segmentation_target target_name main_source)
    add_executable(${target_name} ${main_source})
    target_link_libraries(${target_name}
        PRIVATE
            image_processor image_utils timing_utils litert_cc_api
            absl::log absl::check EGL GLSV3 android log
    )
endmacro()

add_segmentation_target(cpp_segmentation_cpu main_cpu.cc)
add_segmentation_target(cpp_segmentation_gpu main_gpu.cc)
add_segmentation_target(cpp_segmentation_npu main_npu.cc)
```

步驟 7 · 約 10 分鐘

7. 建構 - 選項 B：使用 Bazel 建構 (從來源)

如果您偏好使用 Bazel 做為建構系統 (從來源編譯 LiteRT 執行階段)，或是需要在現有的 Bazel 工作區中工作，請選擇這個路徑。

必要條件

除了「事前準備」一節列出的 NDK 和 ADB 之外，您還需要：

- 已安裝 **Bazel** 並位於 `PATH`。
- LiteRT 範例來源存放區的完整副本。

步驟 1：設定 LiteRT 範例工作區

所有指令都是從 **LiteRT** 範例存放區的根目錄執行

```
cd /path/to/litert-samples
./configure
```

出現提示時：

- 接受 Python 和 Python 程式庫路徑的預設值。
- 對 ROCm 和 CUDA 支援回答「N」。
- 選取「clang」（已使用 18.1.3 測試）做為編譯器。
- 接受預設最佳化旗標。
- 回答 **Y**，為 Android 建構作業設定 WORKSPACE。
- 將最低 Android NDK 級別設為至少 **26**。
- 提供 Android SDK 的路徑。
- 將 Android SDK API 級別設為預設值 (**36**)，並將建構工具設為 **36.0.0**。

```
aravindmurali@my-linux-machine:~/Documents/litert-samples/code-labs/litert-samples$ ./configure
Clang will be used to compile TensorFlow.

Please specify the path to clang executable. [Default is /usr/lib/llvm-19/bin/clang]:

You have Clang 19.1.7 installed.

Please specify optimization flags to use during compilation when bazel option "--config=opt" is specified [Default is -fno-sign-compare]:

Would you like to interactively configure ./WORKSPACE for Android builds? [y/N]: y
Searching for NDK and SDK installations.

Please specify the home path of the Android NDK to use. [Default is /usr/local/google/home/aravindmurali/Android/Sdk/ndk-bundle]: /usr/local/google/home/aravindmurali/Android/Sdk/ndk/25.2.9519653

Please specify the (min) Android NDK API level to use. [Available levels: [16, 17, 18, 19, 21, 22, 23, 24, 26, 27, 28, 29, 30, 31, 32, 33]] [Default is 26]:

Please specify the home path of the Android SDK to use. [Default is /usr/local/google/home/aravindmurali/Android/Sdk]:

Please specify the Android SDK API level to use. [Available levels: ['34', '36']] [Default is 36]:

Please specify an Android build tools version to use. [Available versions: ['34.0.0', '35.0.0', '36.1.0']] [Default is 36.1.0]:

Preconfigured Bazel build configs. You can use any of the below by adding "--config=<name>" to your build command. See .bazelrc for more details.
--config=mkl          # Build with MKL support.
--config=mkl_aarch64  # Build with oneDNN and Compute Library for the Arm Architecture (ACL).
--config=monolithic   # Config for mostly static monolithic build.
--config=numa         # Build with NUMA support.
--config=dynamic_kernels # (Experimental) Build kernels into separate shared objects.
--config=v1           # Build with TensorFlow 1 API instead of TF 2 API.
Preconfigured Bazel build configs to DISABLE default on features:
--config=noqcp        # Disable QCP support.
--config=nonccl       # Disable NVIDIA NCCL support.
Configuration finished
```

步驟 2：建構 CPU 和 GPU 目標

```
# CPU
bazel build \

//compiled_model_api/image_segmentation/c++_segmentation/build_from_source:cpp_segmentation_cpu \
  --config=android_arm64

# GPU
bazel build \

//compiled_model_api/image_segmentation/c++_segmentation/build_from_source:cpp_segmentation_gpu \
  --config=android_arm64
```

步驟 3 - 建構 NPU 目標

Qualcomm HTP

1. 下載 QAI RT SDK 2.41 以上版本並解壓縮。
2. 確認解壓縮的 SDK 內容位於名為 `latest/` 的子目錄中：

```
/path/to/qairt_sdk/
├── latest/
│   ├── include/
│   ├── lib/
│   └── ...
```

3. 建構時，請傳遞以 `/` 結尾的父項路徑：

```
bazel build \

//compiled_model_api/image_segmentation/c++_segmentation/build_from_source:cpp_segmentation_npu \
  --config=android_arm64 \
  --nocheck_visibility \
  --action_env LITERT_QAI RT_SDK=/path/to/qairt_sdk/
```

由於部分上游 LiteRT 目標的預設瀏覽權限受到限制，因此建構時需要加上 `--nocheck_visibility` 標記。

MediaTek APU

不需要額外 SDK。NeuroPilot 執行階段是 Dimensity 9400 裝置上的系統程式庫。

```
bazel build \

//compiled_model_api/image_segmentation/c++_segmentation/build_from_source:cpp_segmentation_npu_mtk \
  --config=android_arm64 \
  --nocheck_visibility
```

```
aravindmurali@linux-machine:~/Documents/litert-samples/codelabs/litert-samples$ bazel build //compiled_model_api/image_segmentation/c++_segmentation/build_from_source:cpp_segmentation_cpu --config=android_arm64
INFO: Reading 'startup' options from /usr/local/google/home/aravindmurali/Documents/litert-samples/codelabs/litert-samples/.bazelrc: --window_enable_symlinks
INFO: Options provided by the client:
  Inherited 'common' options: --isatty=1 --terminal-columns=258
INFO: Reading rc options for 'build' from /usr/local/google/home/aravindmurali/Documents/litert-samples/codelabs/litert-samples/.bazelrc:
  Inherited 'common' options: --experimental_repo_remote_exec --experimental_repo_remote_exec --noenable_bzlmod
INFO: Reading rc options for 'build' from /usr/local/google/home/aravindmurali/Documents/litert-samples/codelabs/litert-samples/.bazelrc:
  'build' options: --incompatible_enable_cc_toolchain_resolution --repo env USE HERMETIC CC TOOLCHAIN=1 --repo env USE PIPWRAP RULES=True --copt=DGRPC_BAZEL_BUILD --cxdopt=stdcgnu++17 --host_copt=DGRPC_BAZEL_BUILD --action env=GRPC_BAZEL_RUNTIME=1 --repo env=PROTOCOL_BUFFERS_PYTHON_IMPLEMENTATION=upb --action env=PROTOCOL_BUFFERS_PYTHON_IMPLEMENTATION=upb --rules python/python/config/settings:precompile=force disabled --define framework_shared_object=true --define tsl_protobuf_header_only=true --define=allo
w_oversize_protos=true --spawn_strategy=standalone -c opt --announce_rc --define=grpc_no_ares=true --no incompatible_remove_legacy_whole_archive --features=force_no_whole_archive --host_features=force_no_whole_archive --enable_platform_specific_config --def
inewith_xla_support=true --config=short_logs --experimental_cc_shared_library --experimental_link_static_libraries_once=false --incompatible_enforce_config_setting_visibility
INFO: Reading rc options for 'build' from /usr/local/google/home/aravindmurali/Documents/litert-samples/codelabs/litert-samples/.litert_configure.bazelrc:
  'build' options: --action env PYTHON_BIN_PATH=/usr/bin/python3 --action env PYTHON_LIB_PATH=/usr/lib/python3.13/dist-packages --python path=/usr/bin/python3 --action env CLANG_COMPILER_PATH=/usr/lib/llvm-19/bin/clang --repo env=cc=/usr/lib/llvm-19/bin/clang
g --repo env=BAZEL_COMPILER=/usr/lib/llvm-19/bin/clang --action env ANDROID_NDK_HOME=/usr/local/google/home/aravindmurali/Android/Sdk/ndk/25.2.9519653 --action env ANDROID_NDK_API_LEVEL=26 --action env ANDROID_BUILD_TOOLS
VERSION=36.1.0 --action env ANDROID_SDK_API_LEVEL=38 --action env ANDROID_SDK_HOME=/usr/local/google/home/aravindmurali/Android/Sdk
INFO: Reading rc options for 'build' from /usr/local/google/home/aravindmurali/Documents/litert-samples/codelabs/litert-samples/.bazelrc.user:
  'build' options: --incompatible_enable_apple_toolchain_resolution --apple_crosstool_top=@build_bazel_apple_support/crosstool:toolchain --host_crosstool_top=@build_bazel_apple_support/crosstool:toolchain --action env=DEVELOPER_DIR=/Applications/Xcode.app/Contents/Developer --action env=SDKROOT=/Applications/Xcode.app/Contents/Developer/Platforms/iPhoneSimulator.platform/Developer/SDKs/iPhoneSimulator.sdk
INFO: Reading rc options for 'build' from /usr/local/google/home/aravindmurali/Documents/litert-samples/codelabs/litert-samples/.bazelrc:
  'build' options: --config=v2 --config=litert_prefixes --define=disable_tf_lite_pytrue
INFO: Found applicable config definition build:short_logs in file /usr/local/google/home/aravindmurali/Documents/litert-samples/codelabs/litert-samples/.bazelrc: --output_filter=DONT_MATCH_ANYTHING
INFO: Found applicable config definition build:v2 in file /usr/local/google/home/aravindmurali/Documents/litert-samples/codelabs/litert-samples/.bazelrc: --define=tf_api_version=2 --action env=TF_BEHAVIOR=1
INFO: Found applicable config definition build:litert_prefixes in file /usr/local/google/home/aravindmurali/Documents/litert-samples/codelabs/litert-samples/.bazelrc: --define=litert_prefixes=external/org_tensorflow/
INFO: Found applicable config definition build:android_arm64 in file /usr/local/google/home/aravindmurali/Documents/litert-samples/codelabs/litert-samples/.bazelrc: --config=android --cpu=arm64-v8a --fat_apk_cpu=arm64-v8a --platforms=@org_tensorflow/tensorflow/tools/toolchains/android_arm64_v8a
INFO: Found applicable config definition build:android in file /usr/local/google/home/aravindmurali/Documents/litert-samples/codelabs/litert-samples/.bazelrc: --config=clang_local --crosstool_top=/external:android/crosstool --host_crosstool_top=@bazel_tools
//tools/cpp:toolchain --dynamic_mode=off --define=xnn_enable_avxvnniint8=false --noenable_platform_specific_config --copt=-w --cxdopt=stdc++17 --host_cxdopt=stdc++17 --definewith_xla_support=false
INFO: Found applicable config definition build:clang_local in file /usr/local/google/home/aravindmurali/Documents/litert-samples/codelabs/litert-samples/.bazelrc: --no incompatible_enable_cc_toolchain_resolution --no incompatible_enable_android_toolchain_resol
ution --repo env USE HERMETIC CC TOOLCHAIN=1
INFO: Analyzed target //compiled_model_api/image_segmentation/c++_segmentation/build_from_source:cpp_segmentation_cpu (2 packages loaded, 12 targets configured).
INFO: Found 1 target...
Target //compiled_model_api/image_segmentation/c++_segmentation/build_from_source:cpp_segmentation_cpu up-to-date:
  bazel-bin/compiled_model_api/image_segmentation/c++_segmentation/build_from_source/cpp_segmentation_cpu
INFO: Elapsed time: 37.562s, Critical Path: 29.51s
INFO: 1798 processes: 4 internal, 1786 local.
INFO: Build completed successfully, 1798 total actions
```

BUILD 檔案

開啟 `build_from_source/BUILD`。這會定義四個 `cc_binary` 目標 (每個加速器各一個，加上專用的 MediaTek NPU 目標)，每個目標都依附於共用的 `image_processor`、`image_utils` 和 `timing_utils` 程式庫目標：

```
cc_binary(
    name = "cpp_segmentation_cpu",
    srcs = ["main_cpu.cc"],
    deps = [
        ":image_processor",
        ":image_utils",
        ":timing_utils",
        "@litert_archive//litert/cc:litert_api_with_dynamic_runtime",
        "@com_google_absl//absl/time",
        "@com_google_absl//absl/types:span",
    ] + gles_deps() + gl_native_deps(),
    ...
)
```

GPU 目標會將 `libLiteRtClGlaAccelerator.so` 新增為資料依附元件，因此 Bazel 會將其納入執行檔。NPU 目標會將供應商調度器和編譯器外掛程式 `.so` 檔案新增為資料依附元件。

步驟 8 · 約 8 分鐘

8. 使用運算著色器進行 GPU 加速前處理

這三個進入點都使用相同的 OpenGL ES 運算著色器管道進行前處理。瞭解這項概念是瞭解 GPU 路徑為何比 CPU 路徑快上許多的重要關鍵。

設定無頭 EGL 環境

`ImageProcessor::InitializeGL()` 會建立無頭 EGL 情境，也就是沒有附加視窗或螢幕的 OpenGL 情境。這是 Android 螢幕外 GPU 計算的標準做法。接著，從磁碟編譯五個 GLSL 運算著色器程式：

```
processor.InitializeGL(
    "shaders/passthrough_shader.vert",
    "shaders/mask_blend_compute.glsl",
    "shaders/resize_compute.glsl",
    "shaders/preprocess_compute.glsl",
    "shaders/deinterleave_masks.glsl");
```

將輸入圖片上傳至 GPU

JPEG 會由 `ImageUtils::LoadImage()` (透過 STB 程式庫) 解碼至 CPU 記憶體，然後上傳至 GPU 紋理：

```
auto img_data_cpu = ImageUtils::LoadImage(
    input_file, width_orig, height_orig, channels_file, /*desired=*/3);

GLuint tex_id_orig = processor.CreateOpenGLTexture(
    img_data_cpu, width_orig, height_orig, loaded_channels);

ImageUtils::FreeImageData(img_data_cpu); // CPU copy no longer needed
```

從這個時間點開始，原始圖片會以 OpenGL 紋理的形式存在於 GPU 記憶體中。

預先處理運算著色器

`shaders/preprocess_compute.glsl` 會在 256×256 的輸出格線中，調度 8×8 的執行緒群組。每個執行緒會處理一個輸出像素：使用雙線性濾鏡 (免費的硬體調整大小) 對輸入紋理進行取樣、將 `[0, 1]` RGB 值轉換為 `[-1, 1]`，然後寫入輸出 SSBO：

```
vec2 uv = vec2(float(pos.x) / float(out_width - 1),
               float(pos.y) / float(out_height - 1));
vec4 color_0_1 = texture(inputTexture, uv);
vec3 color_neg1_1 = (color_0_1.rgb * 2.0) - 1.0;

int base = (pos.y * out_width + pos.x) * num_channels;
preprocessed_output.data[base + 0] = color_neg1_1.r;
preprocessed_output.data[base + 1] = color_neg1_1.g;
preprocessed_output.data[base + 2] = color_neg1_1.b;
```

如果是標準 (非零複製) 路徑，這個 SSBO 隨後會讀回 CPU，並寫入 LiteRT 張量：

```
std::vector<float> preprocessed(256 * 256 * num_channels);
processor.ReadBufferData(preprocessed_buffer_id, 0,
                        preprocessed.size() * sizeof(float),
                        preprocessed.data());
LITERT_ABORT_IF_ERROR(
    input_buffers[0].Write(absl::MakeConstSpan(preprocessed)));
```

步驟 9 · 約 5 分鐘

9. CPU 推論

開啟 `main_cpu.cc` 。LiteRT 設定為三行：

```
// Create the root environment
LITERT_ASSIGN_OR_ABORT(auto env, litert::Environment::Create({}));

// Compile the model for the CPU
LITERT_ASSIGN_OR_ABORT(auto compiled_model,
    litert::CompiledModel::Create(
        env, model_path, litert::HwAccelerators::kCpu));

// Allocate input and output tensor buffers
LITERT_ASSIGN_OR_ABORT(auto input_buffers,
    compiled_model.CreateInputBuffers());
LITERT_ASSIGN_OR_ABORT(auto output_buffers,
    compiled_model.CreateOutputBuffers());
```

預先處理後，推論會是單一同步呼叫：

```
LITERT_ABORT_IF_ERROR(compiled_model.Run(input_buffers, output_buffers));
```

`Run()` 會封鎖，直到推論完成為止。 `selfie_multiclass_256x256.tflite` 浮點模型會在 ARM Cortex 核心上執行，在中階裝置上通常需要約 116 到 128 毫秒。

二進位檔用法：

```
cpp_segmentation_cpu <model_path> <input_image> <output_image>
```

10. GPU 推論 (OpenCL)

開啟 `main_gpu.cc`。GPU 路徑會介紹 CPU 路徑中沒有的兩個概念：`litert::Options` (用於設定 GPU 加速器，搭配 OpenCL 後端) 和非同步執行。

設定 GPU 選項

```
litert::Options CreateGpuOptions() {
    LITERT_ASSIGN_OR_ABORT(litert::Options options, litert::Options::Create());
    LITERT_ASSIGN_OR_ABORT(auto& gpu_options, options.GetGpuOptions());

    LITERT_ABORT_IF_ERROR(
        gpu_options.SetBackend(litert::GpuOptions::Backend::kOpenCl));

    // Allow CPU fallback for any ops not supported by the GPU delegate
    options.SetHardwareAccelerators(litert::HwAccelerators::kGpu |
                                    litert::HwAccelerators::kCpu);

    return options;
}
```

非同步推論

GPU 路徑使用 `RunAsync()`，而非 `Run()`。這會將工作提交至 GPU 指令佇列，並立即傳回。然後同步處理，再讀取結果：

```
bool async = false;
LITERT_ABORT_IF_ERROR(
    compiled_model.RunAsync(0, input_buffers, output_buffers, async));

if (output_buffers[0].HasEvent()) {
    LITERT_ASSIGN_OR_ABORT(auto event, output_buffers[0].GetEvent());
    event.Wait();
}
```

這種非封鎖設計可讓您在即時管道中，將 CPU 工作與 GPU 執行作業重疊。

二進位檔用法：

```
cpp_segmentation_gpu <model_path> <input_image> <output_image>
```

11. 後置處理 - 去交錯和混合

`Run()` 或 `RunAsync()` 完成後，`output_buffers[0]` 會以交錯順序保存形狀 `[256 × 256 × 6]` 的平面浮點陣列。像素 `(row, col)` 的 6 個類別分數位於索引 `(row * 256 + col) * 6` 到 `(row * 256 + col) * 6 + 5`。

將交錯式資料解交錯至 6 個遮罩 SSBO

CPU 輔助程式會將交錯陣列分割為 6 個單一通道浮點陣列，並將每個陣列上傳至各自的 GPU SSBO：

```
std::vector<float> data(256 * 256 * 6);
output_buffers[0].Read(absl::MakeSpan(data));

std::vector<GLuint> mask_ids(6);
for (int i = 0; i < 6; ++i)
    mask_ids[i] = processor.CreateOpenGLBuffer(nullptr, 256 * 256 * sizeof(float));

processor.DeinterleaveMasksCpu(data.data(), 256, 256, mask_ids);
```

將色彩混合遮罩套用至原始圖片

`processor.ApplyColoredMasks()` 會執行 `mask_blend_compute.glsl` 著色器。針對每個輸出像素，系統會找出分數最高的類別 (6 個遮罩 SSBO 的 `argmax`)，並將對應的顏色以 Alpha 合成方式疊加在原始圖片像素上。每個進入點都會定義六種顏色：

```
std::vector<RGBAColor> mask_colors = {
    {1.0f, 0.0f, 0.0f, 0.1f}, // red    - background
    {0.0f, 1.0f, 0.0f, 0.1f}, // green - hair
    {0.0f, 0.0f, 1.0f, 0.1f}, // blue  - body skin
    {1.0f, 1.0f, 0.0f, 0.1f}, // yellow - face skin
    {1.0f, 0.0f, 1.0f, 0.1f}, // magenta - clothes
    {0.0f, 1.0f, 1.0f, 0.1f}, // cyan   - accessories
};
```

`0.1f` 的 Alpha 值會讓色調保持細緻，因此原始圖片仍會顯示。

儲存輸出內容

最後會讀回混合的 RGBA 浮點 SSBO，並限制取值範圍為 `[0, 1]`、轉換為 `unsigned char`，然後儲存為 PNG：

```
for (size_t i = 0; i < float_data.size(); ++i)
    uchar_data[i] = static_cast<unsigned char>(
        std::max(0.0f, std::min(1.0f, float_data[i])) * 255.0f);
ImageUtils::SaveImage(output_file, width, height, 4, uchar_data.data());
```

步驟 12 · 約 5 分鐘

12. 在裝置上部署及執行

使用 USB 連接 Android 裝置，並確認 ADB 連線：

```
adb devices
```

```
aravindmurali@my-linux-machine:~/Documents/litert-samples/codelabs/litert-samples$ adb devices
List of devices attached
R5CWB0Y2R8V    device
```

使用 `deploy_and_run_on_android.sh`

每個變體都有自己的部署指令碼。CMake 變體會指向 `build/` 目錄，Bazel 變體則會指向 `bazel-bin/`。兩個指令碼：

1. 在裝置上建立 `/data/local/tmp/cpp_segmentation_android/`。
2. 推送二進位檔、GLSL 著色器、模型、測試圖片和執行階段 `.so` 檔案。
3. 使用 `adb shell` 執行推論。
4. 將 `output_segmented.png` 拉回裝置。

CMake 變體 (`use_prebuilt_litert/`)

```
# CPU
./deploy_and_run_on_android.sh --accelerator=cpu --phone=s25 build/

# GPU
./deploy_and_run_on_android.sh --accelerator=gpu --phone=s25 build/
```

Bazel 變體 (`build_from_source/`)

從 LiteRT 範例存放區根目錄執行下列指令：

```
# CPU
./compiled_model_api/image_segmentation/c++_segmentation/build_from_source/deploy_and_run_on_
android.sh \
    --accelerator=cpu --phone=s25 bazel-bin/

# GPU
./compiled_model_api/image_segmentation/c++_segmentation/build_from_source/deploy_and_run_on_
android.sh \
    --accelerator=gpu --phone=s25 bazel-bin/
```

`--phone` 旗標可控管要使用哪些裝置專屬模型和供應商程式庫。支援的值：`s24` (Snapdragon 8 Gen 3)、`s25` (Snapdragon 8 Elite)、`dim9400` (MediaTek Dimensity 9400)、`pixel8` (Tensor G3)、`pixel9` (Tensor G4)、`pixel10` (Tensor G5) 和 `pixel11` (Tensor G6)。

推論時間

推論完成後，`PrintTiming()` 會列印完整的剖析細目：

Load image: X ms
Preprocess: X ms
Inference: X ms
Postprocess: X ms
E2E: X ms
Save image: X ms

Samsung S25 Ultra (Snapdragon 8 Elite) 的參考效能：

加速器	執行作業類型	推論	E2E
CPU	同步	約 116 至 128 毫秒	~157 毫秒
GPU (OpenCL)	非同步	~0.95 毫秒	約 35 至 43 毫秒

13. 進階 (選用)：NPU 推論

為達到最高效能，LiteRT 支援使用供應商專屬外掛程式庫的 NPU 加速功能。NPU 路徑可將端對端延遲時間降至 9 毫秒。

支援的裝置和模式

方塊	裝置範例	模式	E2E
高通 SM8650	Galaxy S24	AOT	約 17 毫秒
高通 SM8750	Galaxy S25	AOT	約 17 毫秒
Qualcomm (不限)	—	JIT	約 28 毫秒
MediaTek Dimensity 9400	—	JIT	~9 毫秒
Google Tensor G3-G6	Pixel 8 到 Pixel 11	AOT/JIT	因解決方案而異

AOT (預先) 使用裝置專用的預先編譯模型 (例如 `selfie_multiclass_256x256_SM8650.tflite`)。這是最快的選項，但僅適用於特定晶片。

JIT (即時) 使用標準 `selfie_multiclass_256x256.tflite`，並在執行階段編譯至 NPU，首次執行速度較慢，但與晶片無關。

其他必要條件

Qualcomm HTP：

- QAIRT SDK 2.41 以上版本 (提供 `libQnnHtp.so`、存根或骨架 `.so` 檔案)。
- `libLiteRtDispatch_Qualcomm.so`，位於 GitHub 上的 LiteRT NPU 執行階段程式庫版本中。

MediaTek APU：

- `libLiteRtDispatch_MediaTek.so`。
- NeuroPilot 執行階段 (已是 Dimensity 9400 裝置上的系統程式庫，無須推送)。

Google Tensor：

- `libLiteRtDispatch_GoogleTensor.so`。

NPU 環境和選項

`main_npu.cc` 將 `Environment` 指向裝置上的供應商調度程式庫目錄，然後設定供應商專屬的效能選項：


```
// Configure LiteRT to find the dispatch library
std::vector<litert::Environment::Option> env_opts;
env_opts.push_back({litert::Environment::OptionTag::DispatchLibraryDir,
                    kQualcommDispatchDir});
LITERT_ASSIGN_OR_ABORT(auto env,
                        litert::Environment::Create(std::move(env_opts)));

// Target NPU with CPU fallback
LITERT_ASSIGN_OR_ABORT(litert::Options options, litert::Options::Create());
options.SetHardwareAccelerators(litert::HwAccelerators::kNpu |
                                litert::HwAccelerators::kCpu);

// Qualcomm: burst performance mode
auto& qnn_opts = options.GetQualcommOptions();
qnn_opts.SetLogLevel(litert::qualcomm::QualcommOptions::LogLevel::kOff);
qnn_opts.SetHtpPerformanceMode(
    litert::qualcomm::QualcommOptions::HtpPerformanceMode::kBurst);

LITERT_ASSIGN_OR_ABORT(auto model,
                        litert::CompiledModel::Create(env, model_path, options));
```

如果是 MediaTek，請取代 `GetQualcommOptions()` 區塊：

```
// MediaTek: fast single-answer mode + low-latency hint
auto& mtk_opts = options.GetMediatekOptions();
mtk_opts.SetPerformanceMode(
    litert::mediatek::MediatekOptions::PerformanceMode::kFastSingleAnswer);
mtk_opts.SetOptimizationHint(
    litert::mediatek::MediatekOptions::OptimizationHint::kLowLatency);
mtk_opts.SetNeronSDKVersionType(
    litert::mediatek::MediatekOptions::NeronSDKVersion::kVersion8);
```

部署至 NPU

CMake 變種版本 - Qualcomm S25 (AOT)

```
./deploy_and_run_on_android.sh \
    --accelerator=npu --phone=s25 \
    --host_npu_lib=/path/to/qairt/lib \
    --host_npu_dispatch_lib=/path/to/dir/with/libLiteRtDispatch_Qualcomm.so \
    build/
```

CMake 變體 - MediaTek Dimensity 9400 (JIT)

```
./deploy_and_run_on_android.sh \
    --accelerator=npu --phone=dim9400 --jit \
    --host_npu_dispatch_lib=/path/to/dir/with/libLiteRtDispatch_MediaTek.so \
    build/
```

Bazel 變種版本 - Qualcomm S25 (AOT)

```
./compiled_model_api/image_segmentation/c++_segmentation/build_from_source/deploy_and_run_on_
android.sh \
    --accelerator=npu --phone=s25 bazel-bin/
```

Bazel 變體 - MediaTek Dimensity 9400 (JIT)

```
./compiled_model_api/image_segmentation/c++_segmentation/build_from_source/deploy_and_run_on_
android.sh \
    --accelerator=npu --phone=dim9400 --jit bazel-bin/
```

Bazel 變種版本：Google Tensor Pixel 9 (JIT)

```
./compiled_model_api/image_segmentation/c++_segmentation/build_from_source/deploy_and_run_on_
android.sh \
    --accelerator=npu --phone=pixel9 --jit bazel-bin/
```

如果是 Bazel 變體，當 `LITERT_QAIRT_SDK` 在建構時間設定，系統會自動從 `bazel-bin` 執行檔樹狀結構中挑選 QAIRT SDK 程式庫。CMake 變數需要 `--host_npu_lib` 旗標，才能指向已解壓縮的 QAIRT SDK。

14. 恭喜！

您已成功使用 LiteRT，在 Android 上建構及執行 C++ 影像分割管道。您已學會如何：

- 使用 CMake + NDK 或 Bazel，為 Android `arm64-v8a` 交叉編譯 C++ 二進位檔。
- 使用 LiteRT C++ API (`Environment` 、 `CompiledModel` 、 `TensorBuffer`) 進行高效的裝置端推論。
- 使用 OpenGL ES 3.1 運算著色器，在 GPU 上預先處理圖像資料。
- 執行同步 CPU 推論，以及非同步 GPU (OpenCL) 推論。
- 為 Qualcomm、MediaTek 和 Google Tensor 裝置設定 NPU 加速功能。
- 使用 ADB 在 Android 上部署及執行 C++ 二進位檔。

後續步驟

- 換用其他 TFLite 模型 (例如深度估算或姿勢偵測)。
- 使用 JNI 將 C++ 管道整合至 Android NDK 應用程式。
- 使用 Android GPU 檢查器剖析記憶體用量，並輸出時間資訊。
- 探索模型量化，進一步縮短 NPU 推論延遲時間。

瞭解詳情

- [LiteRT 說明文件](#)
 - [LiteRT C++ SDK 指南](#)
 - [LiteRT GitHub](#)
 - [LiteRT 範例 GitHub](#)
-